

CARTRIDGE & CLOUD · DOCUMENTACIÓN RC1

Cartridge & Cloud — UX Flow

Edición PDF con identidad visual unificada de VRM Games. El contenido procede íntegramente del archivo Markdown original.

PROYECTO

Cartridge & Cloud

ESTADO

Fuente vigente de experiencia de usuario y flujos

DOCUMENTO

05 / 05_UX_Flow.md

AUTORÍA

VRM Games / Blas Luis Rocha González

VERSIÓN

1.0

FECHA DOCUMENTAL

2026-07-01

Control y metadatos del documento

Archivo fuente: 05_UX_Flow.md
SHA-256 del Markdown: e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e
Tamaño del Markdown: 56,805 bytes
Conversión: representación visual completa; el Markdown original mantiene la autoridad sobre el texto fuente.

Front matter original

title	Cartridge & Cloud — UX Flow
author	VRM Games / Blas Luis Rocha González
date	2026-07-01
lang	es-ES
document_version	1.0
status	Fuente vigente de experiencia de usuario y flujos
project	Cartridge & Cloud
platform	PC / Steam
engine	Unity 6.3 LTS 6000.3.18f1
render_pipeline	URP 17.3.0
application_version_reference	0.0.17

Índice

Cartridge & Cloud — UX Flow

- 0. Propósito del documento
 - 0.1. Jerarquía de autoridad
 - 0.2. Estados de madurez
 - 0.3. Principio de amplitud
- 1. Objetivos de experiencia
 - 1.1. Objetivo principal
 - 1.2. Objetivos de claridad
 - 1.3. Objetivos de control
 - 1.4. Objetivos de aprendizaje
 - 1.5. Objetivos de accesibilidad
- 2. Arquitectura global de navegación
 - 2.1. Flujo de producción
 - 2.2. Contrato de Bootstrap
 - 2.3. Mapa de alto nivel
 - 2.4. Reglas de navegación
- 3. Estados globales de interacción
 - 3.1. Estados principales
 - 3.2. Prioridad de consumo
 - 3.3. Regla del puntero sobre UI
 - 3.4. Escape y cancelación
- 4. Modelo de input de referencia
 - 4.1. Ratón
 - 4.2. Teclado
 - 4.3. Remapeo
 - 4.4. Input durante pausa
- 5. Arranque y MainMenu
 - 5.1. Arranque correcto
 - 5.2. MainMenu
 - 5.3. Estado sin guardados
 - 5.4. Estado con guardados
 - 5.5. Salir de la aplicación
- 6. Flujo de slots y persistencia
 - 6.1. Slot vacío
 - 6.2. Slot válido
 - 6.3. Sustituir partida
 - 6.4. Borrar
 - 6.5. Recuperación desde backup
 - 6.6. Guardado manual
 - 6.7. Autoguardado
- 7. Entrada a StoreInitial
 - 7.1. Carga de escena
 - 7.2. Primer frame jugable
 - 7.3. Error de composición
 - 7.4. Autoría visible
- 8. HUD y jerarquía de información
 - 8.1. Información permanente
 - 8.2. Información contextual
 - 8.3. Operations UI
 - 8.4. Densidad de información
- 9. Golden Path del vertical slice
 - 9.1. Recorrido completo
 - 9.2. Criterios del Golden Path
 - 9.3. Primeros treinta minutos
- 10. Flujo de movimiento y cámara
 - 10.1. Movimiento por clic
 - 10.2. Destino inválido
 - 10.3. Cámara orbital

10.4. Obstrucción

11. Flujo de interacción de mundo

11.1. Descubrimiento

11.2. Selección

11.3. Distancia

11.4. Conflictos

12. Modo construcción

12.1. Entrada

12.2. Selección de definición

12.3. Preview

12.4. Feedback de validez

12.5. Rotación

12.6. Confirmación

12.7. Movimiento de objeto existente

12.8. Retirada

12.9. Salida

13. Proveedores y pedidos

13.1. Abrir catálogo

13.2. Crear pedido

13.3. Errores

13.4. Estado de pedido

13.5. Llegada

14. Recepción e inventario

14.1. Recibir entrega

14.2. Inventario

14.3. Transferencia

14.4. Estados vacíos

15. Displays, asignación y reposición

15.1. Asignar producto

15.2. Reasignar

15.3. Reponer

15.4. Reposición rápida

15.5. Stock visible

16. Precios

16.1. Consulta

16.2. Edición

16.3. Feedback

17. Flujo de jornada

17.1. Estados

17.2. BeforeOpen

17.3. Apertura

17.4. Open

17.5. Aviso de cierre

17.6. Closing

17.7. Closed

17.8. Results

17.9. Siguiente día

18. Clientes

18.1. Recorrido base

18.2. Legibilidad de intención

18.3. Feedback contextual

18.4. Paciencia

18.5. Reserva temporal

18.6. Abandono

19. Cola y checkout

19.1. Entrada en cola

19.2. Estación

19.3. Checkout

19.4. Preflight

19.5. Fallo

19.6. Feedback de venta

20. Economía y resumen

20.1. HUD económico

20.2. Ledger

20.3. Compras

20.4. Resumen diario

20.5. Pérdida	26.6. Cierre incompleto
21. Pausa, opciones y salida	27. Flujos diferidos cercanos
21.1. Pausa	27.1. Empleados
21.2. Volver a MainMenu	27.2. Investigación
21.3. Opciones	27.3. Puestos informáticos
21.4. Aplicación de cambios	27.4. Reservas comerciales
22. Accesibilidad	28. Flujos empresariales futuros
22.1. Visual	28.1. Comercio online
22.2. Auditiva	28.2. Publishing
22.3. Motora	28.3. Desarrollo interno
22.4. Cognitiva	28.4. Plataforma digital
22.5. Velocidad temporal	28.5. Infraestructura
23. Localización ES/EN	28.6. Mercado y competidores
23.1. Reglas	29. Arquitectura de pantallas
23.2. Cambio de idioma	30. Criterios de aceptación UX
23.3. Terminología base	30.1. Arranque y escenas
24. Tutorial y ayuda	30.2. Slots
24.1. Principios	30.3. Input
24.2. Tutorial inicial	30.4. Construcción
24.3. Ayuda contextual	30.5. Pedidos e inventario
24.4. Ayuda persistente	30.6. Jornada y clientes
25. Mensajes, feedback y diálogos	30.7. Checkout y economía
25.1. Niveles	30.8. Guardado y accesibilidad
25.2. Estructura de error	30.9. Golden Path y build
25.3. Acciones destructivas	31. Telemetría y playtest
25.4. Toasts	31.1. Métricas recomendadas
25.5. Estados de carga	31.2. Preguntas de playtest
26. Fallos y recuperación	31.3. Evidencia
26.1. Guardado inválido	32. Severidades UX
26.2. Escena no cargable	33. Definition of Done de un flujo UX
26.3. Estado incompatible	34. Gates de Sprint 16 y Sprint 17
26.4. Operación rechazada	34.1. Sprint 16 — presentación representativa
26.5. Cliente bloqueado	34.2. Sprint 17 — estabilización

34.3. Condición para iniciar sistemas posteriores

Anexo A. Decisiones sustituidas

Anexo B. Trazabilidad de fuentes

B.1. Comparación de las dos copias v0.3

Anexo C. Regla de mantenimiento

Cartridge & Cloud — UX Flow

0. Propósito del documento

Este documento define cómo una persona entra en *Cartridge & Cloud*, comprende su estado, realiza acciones, recibe feedback, resuelve errores, guarda su progreso y completa el recorrido jugable del vertical slice.

Su función es convertir las reglas del GDD, los requisitos del vertical slice, la arquitectura del TDD y el modelo de datos en **recorridos observables y verificables**.

El documento especifica:

- jerarquía de información;
- navegación entre escenas y pantallas;
- estados globales de interacción;
- propiedad del input;
- recorridos de nueva partida, continuación, sustitución y borrado;
- movimiento, cámara e interacción 3D;
- construcción y autoría funcional del espacio;
- operación diaria;
- pedidos, recepción, inventario, displays y precios;
- clientes, reservas, cola y checkout;
- economía y resumen diario;
- guardado, backup y recuperación;
- tutorial, ayuda y accesibilidad;
- mensajes, confirmaciones, errores y estados vacíos;
- Golden Path del vertical slice;
- criterios de aceptación por flujo;
- recorridos diferidos y futuros.

Este archivo no sustituye al GDD ni al TDD:

- el GDD define reglas y resultados;

- el TDD define responsabilidades técnicas;
- el modelo de datos define propiedad y estructura del estado;
- este UX Flow define lo que la persona ve, entiende y hace en cada momento.

0.1. Jerarquía de autoridad

La consolidación aplica el siguiente orden:

1. `00_Enfoque_y_Alcance.md`
2. `01_Game_Design_Document.md`
3. `02_Vertical_Slice_Specification.md`
4. `03_Technical_Design_Document.md`
5. `04_Modelo_de_Datos.md`
6. `Cartridge_And_Cloud_UX_Flow_v0.5.md`
7. `Cartridge_And_Cloud_UX_Flow_v0.4.md`
8. `Cartridge_And_Cloud_UX_Flow_v0.3.md` de baseline v0.4
9. copia v0.3 de baseline v0.3, solo para trazabilidad
10. `ADR-0009_Bootstrap_Owned_Scene_Flow.md` como decisión técnica complementaria

Cuando exista una contradicción:

- prevalece el recorrido compatible con el vertical slice vigente;
- `StoreInitial.unity` sustituye cualquier tienda conceptual antigua de `5 × 5 m`;
- la UI no puede introducir una mutación que el dominio rechace;
- el clic consumido por UI no puede producir acciones de mundo;
- el estado persistido prevalece sobre una representación visual reconstruida;
- las acciones destructivas requieren confirmación cuando la pérdida no sea trivial;
- los sistemas diferidos se documentan, pero no aparecen como opciones activas antes de su gate.

0.2. Estados de madurez

Estado	Significado
IMPLEMENTADO	Existe recorrido funcional según la baseline v0.6.
OBJETIVO DE SPRINT 16	Debe adquirir presentación representativa y coherencia visual.

Estado	Significado
GATE DE SPRINT 17	Debe estabilizarse, probarse y validarse en build.
DIFERIDO CERCANO	Recorrido definido, pero bloqueado tras el vertical slice.
VISIÓN FUTURA	Flujo de largo plazo sin compromiso de implementación inmediata.
HIPÓTESIS UX	Decisión razonable que requiere playtest o validación de accesibilidad.
SUSTITUIDO	Flujo antiguo que deja de ser normativo.

0.3. Principio de amplitud

La documentación extensa no implica que todas las pantallas deban existir en el mismo sprint. La amplitud protege la coherencia de producto y evita diseñar de forma improvisada cada nueva interfaz.

Los apartados marcados como diferidos deberán permanecer ocultos, deshabilitados o no instanciados hasta que su capacidad esté aprobada.

1. Objetivos de experiencia

1.1. Objetivo principal

La persona debe sentir que opera una tienda física, no una colección de hojas de cálculo.

Cada flujo debe preservar tres capas:

1. **mundo:** dónde están los objetos, personas y recursos;
2. **operación:** qué acción se está realizando;
3. **gestión:** qué datos, costes o consecuencias deben decidirse.

La UI complementa el mundo. No debe sustituir una acción física cuando la representación física aporta comprensión y no introduce fricción innecesaria.

1.2. Objetivos de claridad

En cada momento debe poder responderse:

- ¿Dónde estoy?
- ¿En qué estado está la tienda?
- ¿Qué acción está activa?
- ¿Qué puede hacerse ahora?
- ¿Qué está bloqueado?
- ¿Por qué está bloqueado?
- ¿Qué cambiará al confirmar?
- ¿Cómo se cancela?
- ¿La acción se ha completado?
- ¿Dónde puedo comprobar el resultado?

1.3. Objetivos de control

El juego debe:

- distinguir navegación, interacción, construcción y UI modal;
- no interpretar dos veces el mismo input;
- permitir cancelar antes de una mutación;
- impedir mutaciones parciales;
- comunicar costes antes de confirmar;
- mostrar consecuencias después de confirmar;
- conservar foco y contexto al cerrar un panel;
- evitar que el usuario pierda progreso por errores recuperables;
- permitir pausar la simulación para leer o decidir cuando corresponda.

1.4. Objetivos de aprendizaje

La persona aprende mediante una secuencia:

```
Ver
→ recibir una instrucción breve
→ realizar una acción
→ observar el resultado
→ repetir con menor ayuda
→ combinarla con otras acciones
```

El tutorial no debe explicar sistemas todavía inaccesibles. La ayuda contextual debe aparecer cuando exista una necesidad real.

1.5. Objetivos de accesibilidad

La experiencia debe ser operable sin depender exclusivamente de:

- percepción del color;
- velocidad de reacción;
- lectura de texto pequeño;
- memoria de combinaciones no visibles;
- movimientos bruscos de cámara;
- audición;
- precisión extrema de puntero.

2. Arquitectura global de navegación

2.1. Flujo de producción

```
Inicio de aplicación
→ Bootstrap
→ inicialización de ApplicationRoot
→ validación de servicios mínimos
→ carga asíncrona de MainMenu
→ selección de slot
→ nueva partida o carga
→ carga asíncrona de StoreInitial
→ jornada jugable
→ cierre y resultados
→ guardado
→ siguiente día o MainMenu
→ salida de aplicación
```

TestLab permanece fuera del flujo normal de usuario.

2.2. Contrato de Bootstrap

Bootstrap es la única escena de entrada de producción.

Responsabilidades visibles:

- mostrar una transición controlada si la inicialización no es instantánea;
- evitar una pantalla negra prolongada;
- no exponer herramientas de desarrollo;
- derivar a **MainMenu**;
- mostrar un error recuperable si faltan servicios esenciales.

Responsabilidades técnicas derivadas del ADR-0009:

- **ApplicationRoot** persiste durante cambios de escena;
- la navegación utiliza **LoadSceneAsync** y **LoadSceneMode.Single**;
- las solicitudes concurrentes de transición se rechazan;
- la composición inyecta contratos de navegación en el límite de escena;
- no se crea un service locator genérico;
- no se permite jugar producción entrando directamente en **MainMenu** o **StoreInitial**.

2.3. Mapa de alto nivel

```
MainMenu
├── Slot Flow
│   ├── New Game
│   ├── Continue
│   ├── Replace
│   ├── Delete
│   └── Recover Backup
├── Options
├── Credits
└── Quit

StoreInitial
├── HUD
├── Operations
│   ├── Suppliers
│   ├── Orders
│   ├── Inventory
│   ├── Displays
│   └── Customers
```

```
├── Checkout
├── Day
├── Economy
├── Help
├── Accessibility
├── Build Mode
├── Pause
├── Save
├── Return to MainMenu

Day Flow
├── BeforeOpen
├── Open
├── Closing
├── Closed
├── Results
├── Save
├── Next Day
```

2.4. Reglas de navegación

- Solo una transición de escena puede estar activa.
- Una transición bloquea acciones que muten la partida.
- La pantalla de carga no debe aceptar clics de mundo.
- Volver atrás conserva el estado del panel padre cuando sea útil.
- Cerrar un modal devuelve foco al elemento que lo abrió.
- **Escape** cancela la acción local más profunda antes de abrir pausa.
- Ninguna pantalla destructiva se cierra interpretando el cierre como confirmación.
- La navegación no debe crear guardados implícitos inesperados.
- Salir al menú desde una partida informa si existe progreso no guardado.
- La salida de aplicación debe respetar la política de guardado definida.

3. Estados globales de interacción

3.1. Estados principales

Estado	Propietario del input	Acciones permitidas
Loading	sistema	cancelar solo cuando sea seguro
MainMenuUI	UI	navegación de menús
Exploration	gameplay	mover, cámara, interacción
Interaction	gameplay contextual	completar o cancelar acción
BuildMode	gameplay de construcción	preview, rotación, confirmar, retirar
ModalUI	UI	lectura, edición, confirmación
Paused	UI	opciones, guardar, volver
Closing	sistema + UI limitada	observar resolución, consultar estado
Results	UI	revisar jornada, guardar, continuar
ErrorRecovery	UI	reintentar, recuperar, volver
None	sistema	transición o bloqueo temporal

Solo un estado puede capturar el input principal.

3.2. Prioridad de consumo

Modal de sistema

→ diálogo de confirmación

→ panel UI activo

→ overlay contextual

→ modo construcción

→ interacción de mundo

→ navegación por suelo

Una capa superior que consuma el input impide que las capas inferiores actúen.

3.3. Regla del puntero sobre UI

Antes de lanzar raycast al mundo:

```
if (EventSystem.current != null &&
    EventSystem.current.IsPointerOverGameObject())
{
    return;
}
```

Esta comprobación es obligatoria en cualquier controlador que pueda:

- mover al jugador;
- confirmar placement;
- seleccionar un objeto de mundo;
- abrir una interacción;
- ejecutar checkout;
- retirar mobiliario.

3.4. Escape y cancelación

Orden de resolución de **Escape**:

1. cerrar tooltip fijado;
2. cancelar selector o edición local;
3. cerrar diálogo no destructivo;
4. cancelar preview de construcción;
5. cerrar panel secundario;
6. cerrar Operations;
7. abrir pausa;
8. cerrar pausa y volver a juego.

Un diálogo destructivo no se confirma con **Escape**.

4. Modelo de input de referencia

4.1. Ratón

Entrada	Acción base
Clic izquierdo en suelo	mover al jugador
Clic izquierdo en elemento interactivo	seleccionar o interactuar
Clic izquierdo en UI	ejecutar acción UI
Botón derecho + arrastre	orbitar cámara
Rueda	zoom
Clic izquierdo en BuildMode	confirmar placement válido
Clic derecho en BuildMode	cancelar selección o volver un nivel, según contexto

4.2. Teclado

Entrada	Acción base
B	entrar o salir de BuildMode
Q / E	rotar preview en pasos de 90°
Delete / Backspace	retirar objeto seleccionado, con confirmación cuando proceda
Escape	cancelar o pausa según profundidad
teclas numéricas o accesos	hipótesis futura configurable
navegación UI	teclado y foco cuando el sistema lo soporte

4.3. Remapeo

El remapeo completo es una capacidad planificada. Desde el diseño actual:

- ninguna instrucción debe asumir que una tecla será permanente;
- los textos deben consultar bindings;
- los iconos de input no deben estar incrustados como imágenes fijas;

- los conflictos de bindings deben comunicarse;
- debe existir restauración a valores predeterminados;
- los cambios deben persistir por perfil o configuración local.

4.4. Input durante pausa

La pausa:

- detiene la simulación cuando el estado lo permita;
- mantiene navegación de UI;
- impide mover al personaje;
- impide confirmar placement;
- no altera reservas, colas o transacciones;
- no debe dejar un modal de dominio a medio confirmar.

5. Arranque y MainMenu

5.1. Arranque correcto

```
Ejecutar aplicación
→ mostrar identidad breve o transición
→ inicializar servicios
→ cargar configuración
→ comprobar disponibilidad de guardados
→ abrir MainMenu
→ establecer contexto UI
```

El usuario no debe necesitar interactuar durante Bootstrap.

5.2. MainMenu

Acciones principales:

- seleccionar slot;
- nueva partida;
- continuar;
- opciones;
- créditos;
- salir.

Requisitos:

- el foco inicial es visible;
- la versión de aplicación puede mostrarse discretamente;
- los botones bloqueados explican su estado;
- no se muestra **Continue** global ambiguo cuando existen varios slots;
- el acceso a TestLab no aparece;
- las acciones principales son distinguibles de las destructivas.

5.3. Estado sin guardados

Cuando no existe ningún guardado:

- los tres slots aparecen vacíos;
- cada slot ofrece **Nueva partida**;
- no aparece una acción de continuar inválida;
- puede mostrarse una explicación breve sobre slots;
- el primer foco se sitúa en el primer slot vacío;
- opciones y créditos siguen accesibles.

5.4. Estado con guardados

Cada slot válido muestra, cuando exista información:

- número o nombre del slot;
- día;
- saldo;
- fecha y hora de guardado;

- versión o indicador de compatibilidad;
- miniatura si se aprueba en el futuro;
- estado de recuperación si procede.

Acciones:

- continuar;
- sustituir;
- borrar;
- inspeccionar detalle cuando aporte valor.

5.5. Salir de la aplicación

```
Pulsar Salir  
→ diálogo de confirmación  
→ Confirmar / Cancelar  
→ cierre controlado
```

Desde MainMenu no es necesario guardar partida.

6. Flujo de slots y persistencia

6.1. Slot vacío

```
Seleccionar slot vacío  
→ Nueva partida  
→ confirmar configuración mínima  
→ crear estado inicial  
→ persistir snapshot inicial  
→ cargar StoreInitial
```

La creación no debe dejar un archivo parcial si falla.

6.2. Slot válido

```
Seleccionar slot
→ mostrar resumen
→ Continuar
→ validar snapshot
→ restaurar estado
→ cargar StoreInitial
→ aplicar restauración
→ presentar HUD
```

La carga no se considera completada hasta que la escena y el estado integrado están restaurados.

6.3. Sustituir partida

```
Seleccionar slot ocupado
→ Reemplazar
→ advertir pérdida
→ exigir confirmación explícita
→ crear nueva partida de forma atómica
→ conservar backup si la política lo requiere
```

El botón principal del diálogo debe usar lenguaje inequívoco: **Reemplazar partida**.

6.4. Borrar

```
Seleccionar Borrar
→ mostrar slot, día y fecha
→ confirmar
→ eliminar primario y backup según política
→ actualizar lista
→ mostrar estado vacío
```

El borrado no puede activarse por doble clic accidental.

6.5. Recuperación desde backup

Cuando el primario es inválido y el backup es válido:

```
detectar fallo
→ no sobrescribir archivos
→ informar de la situación
→ ofrecer Recuperar backup
→ restaurar backup
→ marcar recuperación
→ cargar partida
→ recomendar nuevo guardado
```

El mensaje debe distinguir:

- guardado primario corrupto;
- backup disponible;
- fecha del backup;
- posible pérdida de progreso;
- acción recomendada.

6.6. Guardado manual

Desde pausa o estado permitido:

```
Guardar
→ bloquear mutaciones incompatibles
→ capturar IntegratedGameStateSnapshot
→ validar snapshot
→ escribir temporal
→ rotar backup
→ promover primario
→ confirmar éxito
```

Feedback:

- indicador de guardado en curso;
- confirmación breve;
- error persistente si falla;
- nunca mostrar éxito antes de terminar la escritura.

6.7. Autoguardado

El autoguardado puede ejecutarse:

- tras resultados de jornada;
- antes de avanzar al siguiente día;
- en otros puntos aprobados.

No debe:

- interrumpir una transacción;
- generar microcortes graves;
- sobrescribir silenciosamente un estado recuperable con datos inválidos;
- activarse mientras existe una mutación incompleta.

7. Entrada a StoreInitial

7.1. Carga de escena

```
confirmar slot  
→ contexto None  
→ transición visual  
→ cargar StoreInitial asíncronamente  
→ registrar referencias de escena  
→ restaurar estado  
→ validar coherencia mínima  
→ activar contexto Gameplay o UI requerido  
→ retirar transición
```

7.2. Primer frame jugable

Antes de devolver control:

- la tienda está cargada;
- el jugador tiene posición válida;

- la cámara tiene objetivo;
- los muebles persistidos están resueltos;
- el inventario está restaurado;
- los displays conservan asignaciones;
- el día y el dinero son correctos;
- no existe un modal huérfano;
- el HUD muestra datos coherentes;
- no hay movimiento originado por el clic que cerró la carga.

7.3. Error de composición

Si falta una referencia funcional obligatoria:

- no continuar silenciosamente;
- bloquear acciones que puedan corromper estado;
- registrar diagnóstico;
- presentar un mensaje apropiado para build interna;
- permitir volver a MainMenu cuando sea seguro.

7.4. Autoría visible

`StoreInitial.unity` debe comunicar:

- entrada principal;
- mostrador;
- exposición;
- almacenamiento;
- recepción;
- pasillos;
- zonas utilizables;
- zonas bloqueadas;
- puntos de interacción.

La arquitectura no debe depender de marcadores técnicos visibles en la experiencia final.

8. HUD y jerarquía de información

8.1. Información permanente

El HUD debe priorizar:

- día;
- hora;
- estado de tienda;
- dinero;
- acceso a Operations;
- acceso a BuildMode;
- mensajes contextuales;
- estado de guardado cuando se activa.

No debe mostrar todos los subsistemas simultáneamente.

8.2. Información contextual

Aparece cuando es pertinente:

- nombre del objeto;
- acción disponible;
- producto asignado;
- capacidad;
- stock;
- coste;
- motivo de invalidez;
- paciencia de cliente cuando sea legible;
- estado de cola;
- aviso de cierre;
- entrega pendiente.

8.3. Operations UI

Operations centraliza:

- compras;
- proveedores;
- pedidos;
- inventario;
- displays;
- clientes;
- checkout;
- día;
- economía;
- ayuda;
- accesibilidad.

Reglas:

- abrir Operations cambia el contexto a UI o ModalUI;
- el mundo no recibe clics;
- la simulación puede continuar o pausarse según panel y decisión de diseño;
- el panel recuerda la pestaña anterior dentro de la sesión cuando resulte útil;
- cerrar devuelve al contexto previo;
- una pestaña sin datos muestra un estado vacío explicativo.

8.4. Densidad de información

La UI debe revelar detalle progresivamente:

```
resumen  
→ lista  
→ selección  
→ detalle  
→ acción  
→ confirmación
```

No se deben presentar veinte métricas antes de que la persona comprenda la tarea.

9. Golden Path del vertical slice

9.1. Recorrido completo

```
Arrancar
→ elegir slot vacío
→ crear nueva partida
→ cargar StoreInitial
→ aprender movimiento y cámara
→ abrir Operations
→ pedir mercancía
→ esperar o procesar entrega
→ recibir cajas
→ trasladar unidades a almacén
→ asignar producto a display
→ reponer exposición
→ fijar o revisar precio
→ abrir tienda
→ observar entrada de clientes
→ observar búsqueda y reserva
→ resolver cola
→ completar checkout
→ comprobar dinero e inventario
→ cerrar tienda
→ revisar resultados
→ guardar
→ avanzar de día
→ salir a MainMenu
→ cargar el mismo slot
→ comprobar equivalencia
```

9.2. Criterios del Golden Path

- ninguna acción exige consola o herramienta de Editor;
- ningún paso depende de conocimiento tácito;
- los errores explican una solución;
- la UI no mueve al jugador accidentalmente;

- no se duplica stock;
- no se vende una unidad reservada por otro cliente;
- cerrar no deja clientes bloqueados;
- guardar y cargar preserva el resultado;
- la build Windows x64 reproduce el recorrido;
- `Player.Log` no contiene errores bloqueantes.

9.3. Primeros treinta minutos

Secuencia de aprendizaje recomendada:

1. reconocer MainMenu y slots;
2. crear partida;
3. aprender movimiento;
4. aprender cámara;
5. reconocer HUD;
6. abrir Operations;
7. comprender pedido;
8. recibir primera entrega;
9. mover inventario;
10. asignar display;
11. reponer;
12. abrir tienda;
13. observar primer cliente;
14. cobrar primera venta;
15. cerrar;
16. revisar resumen;
17. guardar.

Los tiempos exactos se validan mediante playtest.

10. Flujo de movimiento y cámara

10.1. Movimiento por clic

```
clic en suelo  
→ comprobar UI  
→ raycast  
→ validar superficie  
→ calcular destino  
→ mover  
→ detener en tolerancia
```

Feedback:

- marcador de destino opcional;
- animación coherente;
- ausencia de salto instantáneo;
- fallback si el destino no es alcanzable;
- no repetir mensajes por cada frame.

10.2. Destino inválido

Cuando se hace clic en una superficie no transitable:

- no mover;
- ofrecer feedback discreto cuando sea útil;
- no abrir un error modal;
- no limpiar una selección relevante;
- mantener control de cámara.

10.3. Cámara orbital

- arrastre derecho rota alrededor del jugador;
- la rueda modifica zoom;
- la cámara conserva legibilidad;

- el zoom tiene límites;
- la velocidad es configurable;
- una opción futura reduce movimiento;
- la cámara no debe activar interacciones;
- el cursor no debe quedar capturado fuera de contexto.

10.4. Obstrucción

Cuando una pared u objeto oculta:

- aplicar solución visual aprobada;
- evitar cambios bruscos;
- conservar la relación espacial;
- no volver invisible un objeto interactivo sin feedback;
- validar la solución con el tamaño de `StoreInitial`.

11. Flujo de interacción de mundo

11.1. Descubrimiento

Un objeto interactivo debe comunicar:

- que es seleccionable;
- qué acción principal ofrece;
- si la acción está disponible;
- por qué está bloqueada.

11.2. Selección

```
puntero o proximidad  
→ highlight compatible con estilo
```

```
→ clic  
→ panel contextual o acción
```

El highlight no depende solo del color. Puede combinar contorno, icono, texto o cambio de material moderado.

11.3. Distancia

Si una acción requiere cercanía:

- seleccionar puede ordenar movimiento;
- la acción se ejecuta al llegar;
- si la ruta falla, se cancela con explicación;
- si el objeto cambia de estado, se revalida;
- no se reserva indefinidamente una acción imposible.

11.4. Conflictos

Si un objeto está ocupado o bloqueado:

- mostrar estado;
- no iniciar una segunda operación incompatible;
- permitir cambiar de objetivo;
- evitar esperas ocultas.

12. Modo construcción

12.1. Entrada

```
B o botón Build  
→ comprobar estado de jornada  
→ comprobar modales  
→ abrir catálogo de mobiliario
```

```
→ activar BuildMode  
→ suprimir movimiento
```

Si el modo está bloqueado durante `Open`, debe explicarse. Si se permite, debe definirse el impacto sobre clientes y navegación.

12.2. Selección de definición

Cada elemento muestra:

- nombre;
- familia;
- precio;
- huella;
- capacidad;
- estado de desbloqueo;
- icono o miniatura;
- razón de bloqueo;
- unidades disponibles cuando aplique.

12.3. Preview

```
seleccionar mueble  
→ crear preview  
→ seguir puntero  
→ ajustar a grid  
→ calcular rotación  
→ validar límites  
→ validar solape  
→ validar zona  
→ validar acceso  
→ mostrar resultado
```

12.4. Feedback de validez

Válido:

- tono verde u otro indicador positivo;

- icono de confirmación;
- texto breve opcional;
- coste visible.

Inválido:

- tono rojo;
- patrón, icono o contorno adicional;
- causa concreta:
 - fuera de límites;
 - celdas ocupadas;
 - zona no permitida;
 - bloquea acceso;
 - falta dinero;
 - referencia inválida.

12.5. Rotación

Q y E:

- rotan en pasos de 90°;
- actualizan huella;
- revalidan;
- mantienen punto de anclaje coherente;
- muestran orientación.

12.6. Confirmación

```
clic
→ preflight completo
→ comprobar coste
→ comprobar ocupación
→ aplicar mutación atómica
→ registrar instancia
→ actualizar navegación
→ feedback
```


Un placement inválido no consume dinero ni modifica ocupación.

12.7. Movimiento de objeto existente

Flujo recomendado:

```
seleccionar objeto  
→ Mover  
→ reservar estado anterior  
→ preview en nueva posición  
→ confirmar o cancelar  
→ liberar/ocupar de forma atómica
```

Cancelar devuelve exactamente al estado anterior.

12.8. Retirada

```
seleccionar objeto  
→ Retirar  
→ comprobar contenido o dependencias  
→ advertir consecuencias  
→ confirmar  
→ transferir contenido según regla  
→ liberar celdas  
→ actualizar navegación
```

No debe destruir stock silenciosamente.

12.9. Salida

- **Escape** cancela preview antes de salir;
- **B** cierra el modo cuando no hay operación incompleta;
- se restaura el contexto anterior;
- no se arrastra un clic de cierre al mundo.

13. Proveedores y pedidos

13.1. Abrir catálogo

```
Operations
→ Proveedores
→ seleccionar proveedor
→ ver catálogo
```

Información:

- disponibilidad;
- precio unitario;
- cantidad;
- coste total;
- entrega;
- restricciones;
- saldo.

13.2. Crear pedido

```
añadir líneas
→ modificar cantidades
→ eliminar líneas
→ revisar total
→ comprobar saldo
→ confirmar
→ crear orden
```

El total se recalcula de forma inmediata y exacta.

13.3. Errores

- cantidad no válida;
- catálogo vacío;

- proveedor no disponible;
- saldo insuficiente;
- producto bloqueado;
- error de persistencia.

Cada error debe preservar el carrito de pedido cuando sea seguro.

13.4. Estado de pedido

Estados visibles:

- borrador;
- confirmado;
- pendiente;
- listo para entrega;
- entregado;
- recibido;
- cancelado, si se admite.

No se debe representar `entregado` como `recibido`.

13.5. Llegada

La entrega debe tener presencia física o una representación inequívoca:

- aviso;
 - zona de recepción;
 - cajas;
 - cantidad;
 - proveedor;
 - acción `Recibir`.
-

14. Recepción e inventario

14.1. Recibir entrega

```
seleccionar entrega  
→ inspeccionar líneas  
→ validar destino  
→ confirmar recepción  
→ crear unidades  
→ ubicar en recepción o almacén  
→ cerrar entrega
```

La recepción es idempotente: no puede procesarse dos veces.

14.2. Inventario

El panel distingue:

- recepción;
- almacén;
- exposición;
- reservas temporales;
- otras ubicaciones explícitas.

No debe presentar un total sin permitir conocer su distribución cuando esa distribución afecta a la operación.

14.3. Transferencia

```
seleccionar origen  
→ seleccionar producto  
→ cantidad  
→ seleccionar destino  
→ validar capacidad  
→ confirmar  
→ transferir atómicamente  
→ feedback
```

Errores:

- origen insuficiente;
- destino incompatible;
- capacidad insuficiente;
- cantidad inválida;
- reserva activa;
- estado cambiado durante la operación.

14.4. Estados vacíos

Ejemplos:

- No hay entregas pendientes;
- Este contenedor está vacío;
- No hay unidades disponibles para reponer;
- No existen productos compatibles.

El estado vacío debe explicar la siguiente acción posible.

15. Displays, asignación y reposición

15.1. Asignar producto

```
seleccionar display  
→ Asignar producto  
→ listar compatibles  
→ seleccionar  
→ confirmar  
→ actualizar etiqueta y slots
```

La asignación no crea stock.

15.2. Reasignar

Si existe stock visible:

- advertir;
- ofrecer retirar o transferir;
- impedir pérdida silenciosa;
- conservar la asignación anterior si se cancela.

15.3. Reponer

```
seleccionar display  
→ ver capacidad y stock  
→ Reponer  
→ elegir origen  
→ calcular cantidad máxima  
→ confirmar  
→ transferir unidades  
→ actualizar representación
```

15.4. Reposición rápida

Una acción rápida puede reponer hasta capacidad cuando:

- existe un único origen válido;
- no hay ambigüedad;
- se muestra la cantidad;
- la transferencia sigue siendo atómica.

15.5. Stock visible

La representación debe:

- reflejar cantidad suficiente para comprender disponibilidad;
- no exigir una malla por cada unidad si el coste es excesivo;
- mantenerse sincronizada con el modelo lógico;
- no mostrar productos fantasma después de cargar.

16. Precios

16.1. Consulta

Cada producto muestra:

- coste;
- precio;
- margen;
- referencia de mercado si existe;
- demanda o interés cuando esté disponible;
- stock.

16.2. Edición

```
seleccionar precio  
→ introducir valor  
→ validar  
→ previsualizar margen  
→ confirmar  
→ aplicar
```

Validaciones:

- formato;
- moneda;
- valor mínimo;
- valor máximo razonable;
- redondeo;
- permisos del estado actual.

16.3. Feedback

- no usar solo rojo/verde;
- explicar margen negativo;
- advertir precios extremos;
- no impedir decisiones arriesgadas si el diseño las permite;
- registrar el cambio para persistencia.

17. Flujo de jornada

17.1. Estados

```
BeforeOpen  
→ Open  
→ Closing  
→ Closed  
→ Results  
→ Save  
→ NextDay
```

17.2. BeforeOpen

Permite:

- construir;
- pedir;
- recibir;
- transferir;
- asignar;
- reponer;
- fijar precios;
- revisar economía;

- guardar.

La UI debe mostrar tareas recomendadas sin convertirlas en requisitos artificiales.

17.3. Apertura

```
pulsar Abrir
→ comprobar bloqueos críticos
→ informar si faltan condiciones
→ confirmar
→ cambiar a Open
→ habilitar spawn
→ feedback audiovisual
```

Bloqueos posibles:

- entrada inaccesible;
- estación crítica inválida;
- estado inconsistente.

Advertencias no bloqueantes:

- displays vacíos;
- precios extremos;
- inventario bajo.

17.4. Open

Durante la operación:

- entran clientes;
- se restringen acciones incompatibles;
- se mantiene acceso a información;
- aparecen avisos no intrusivos;
- el jugador resuelve colas y reposición;
- el reloj comunica proximidad de cierre.

17.5. Aviso de cierre

A la hora configurada:

- aviso visual;
- aviso sonoro opcional;
- tiempo restante;
- no bloquear input;
- no repetir de forma molesta.

17.6. Closing

```
iniciar cierre  
→ bloquear spawn  
→ resolver reservas  
→ completar o cancelar operaciones según regla  
→ liberar cola  
→ dirigir clientes a salida  
→ esperar resolución segura
```

No se debe saltar a resultados mientras existan mutaciones pendientes.

17.7. Closed

Permite:

- consultar;
- ordenar;
- recibir si la regla lo permite;
- construir;
- reorganizar;
- guardar;
- pasar a resultados.

17.8. Results

La pantalla muestra:

- ingresos;
- costes;
- beneficio o pérdida;
- ventas;
- abandonos;
- productos destacados;
- incidencias;
- saldo final;
- comparación útil con jornada anterior cuando exista.

17.9. Siguiendo día

```
revisar resultados  
→ continuar  
→ guardar  
→ avanzar día  
→ restablecer estados  
→ volver a BeforeOpen
```

18. Clientes

18.1. Recorrido base

```
spawn  
→ entrada  
→ evaluar objetivos  
→ navegar  
→ buscar producto  
→ reservar unidad  
→ añadir al carrito
```

```
→ repetir o finalizar  
→ cola  
→ checkout  
→ salida
```

Rutas alternativas:

- no encuentra;
- precio inaceptable;
- stock agotado;
- paciencia agotada;
- camino bloqueado;
- tienda cierra;
- error recuperable.

18.2. Legibilidad de intención

El jugador debe poder inferir:

- qué busca el cliente;
- si está satisfecho;
- si espera;
- si no encuentra;
- si abandona;
- si está en cola.

No es necesario mostrar todos los valores numéricos internos.

18.3. Feedback contextual

Medios posibles:

- iconos;
- bocadillos breves;
- animación;
- postura;

- color acompañado de forma;
- panel de inspección;
- mensajes agregados.

18.4. Paciencia

La paciencia debe comunicarse antes del abandono cuando el jugador pueda actuar.

No debe:

- caer sin señal;
- depender de un único color;
- producir abandonos instantáneos injustos;
- exigir perseguir a cada cliente con una pantalla abierta.

18.5. Reserva temporal

Cuando un cliente selecciona una unidad:

- se reserva lógicamente;
- deja de estar disponible para otros;
- el stock visible y UI reflejan el estado cuando sea necesario;
- checkout consume la reserva;
- abandono o cierre la libera;
- guardar y cargar conserva coherencia.

18.6. Abandono

El abandono debe registrar causa:

- sin stock;
- precio;
- espera;
- camino;
- cierre;
- otro motivo definido.

La persona debe poder consultar causas agregadas en resultados o panel de clientes.

19. Cola y checkout

19.1. Entrada en cola

```
cliente finaliza compra  
→ busca estación  
→ reserva posición  
→ entra FIFO  
→ espera
```

La cola debe ser visible y no depender de un contador abstracto.

19.2. Estación

La estación comunica:

- disponible;
- ocupada;
- bloqueada;
- falta operador, si aplica en el futuro;
- cliente actual;
- siguiente cliente.

19.3. Checkout

```
seleccionar estación o cliente  
→ inspeccionar carrito  
→ preflight  
→ confirmar  
→ consumir reservas  
→ registrar venta  
→ mover dinero
```

→ actualizar métricas
→ liberar cliente

19.4. Preflight

Comprueba:

- cliente válido;
- reservas válidas;
- cantidades;
- precios;
- estación;
- estado de jornada;
- ausencia de confirmación previa.

19.5. Fallo

Si falla:

- no retirar unidades;
- no mover dinero;
- no liberar parcialmente reservas;
- mostrar causa;
- permitir reintento cuando proceda;
- registrar diagnóstico.

19.6. Feedback de venta

- total;
- productos;
- ingreso;
- cambio de saldo;
- señal audiovisual;
- actualización de stock;

- avance de cola.

20. Economía y resumen

20.1. HUD económico

Debe mostrar:

- saldo actual;
- variación relevante;
- aviso de coste;
- acceso a detalle.

No es necesario mostrar contabilidad completa permanentemente.

20.2. Ledger

La UI de economía lista movimientos:

- fecha/día;
- tipo;
- cantidad;
- origen;
- referencia;
- saldo resultante cuando exista.

20.3. Compras

Antes de confirmar:

- total;
- saldo actual;
- saldo posterior;

- advertencia si deja poca liquidez;
- bloqueos por saldo insuficiente.

20.4. Resumen diario

Debe responder:

- ¿Cuánto vendí?
- ¿Qué me costó operar?
- ¿Qué beneficio obtuve?
- ¿Qué productos funcionaron?
- ¿Por qué se fueron clientes?
- ¿Qué debo preparar mañana?

20.5. Pérdida

Una jornada negativa:

- no se presenta como error técnico;
- explica principales costes;
- propone información, no una solución obligatoria;
- no oculta movimientos.

21. Pausa, opciones y salida

21.1. Pausa

Acciones:

- continuar;
- guardar;
- opciones;

- ayuda;
- volver a MainMenu;
- salir.

21.2. Volver a MainMenu

```
seleccionar volver  
→ detectar cambios no guardados  
→ ofrecer Guardar y salir / Salir sin guardar / Cancelar  
→ ejecutar opción
```

El orden de botones debe minimizar errores.

21.3. Opciones

Categorías:

- vídeo;
- audio;
- controles;
- idioma;
- accesibilidad;
- gameplay cuando proceda.

21.4. Aplicación de cambios

- cambios seguros pueden aplicarse inmediatamente;
- resolución o pantalla puede requerir confirmación con cuenta atrás;
- cancelar restaura;
- los cambios persistentes se guardan fuera del slot cuando sean globales;
- los cambios por partida se guardan en su lugar correspondiente.

22. Accesibilidad

22.1. Visual

- escala de UI;
- tamaño de texto;
- contraste;
- indicadores no dependientes solo de color;
- contornos legibles;
- reducción de flashes;
- reducción de movimiento de cámara;
- opción de velocidad de cámara;
- cursor visible.

22.2. Auditiva

- subtítulos o equivalentes textuales cuando exista voz;
- señales visuales para avisos importantes;
- controles separados de volumen;
- no depender solo de sonido para cierre, error o venta.

22.3. Motora

- remapeo;
- sensibilidad;
- evitar pulsaciones mantenidas innecesarias;
- tolerancia de clic;
- confirmaciones para acciones destructivas;
- navegación UI con teclado cuando se implemente;
- no exigir acciones de alta velocidad.

22.4. Cognitiva

- lenguaje claro;
- instrucciones breves;
- iconos con texto;
- ayuda contextual;
- historial de avisos importantes;
- estados vacíos explicativos;
- consistencia terminológica;
- pausa para lectura;
- evitar múltiples urgencias simultáneas.

22.5. Velocidad temporal

Cuando se habilite:

- el cambio se comunica;
- no modifica velocidad visual de forma ilegible;
- la pausa no rompe procesos;
- los temporizadores se representan en tiempo de simulación.

23. Localización ES/EN

23.1. Reglas

- ningún texto visible se codifica directamente en lógica;
- claves estables;
- variables parametrizadas;
- plurales;
- formatos de moneda, fecha y número;
- expansión de texto;

- revisión de truncamiento;
- términos de gameplay consistentes.

23.2. Cambio de idioma

Puede requerir:

- actualización inmediata de UI;
- reconstrucción de ciertos paneles;
- persistencia en configuración;
- aviso si una escena debe recargarse.

23.3. Terminología base

Debe existir un glosario para:

- stock;
 - inventario;
 - exposición;
 - reserva;
 - pedido;
 - entrega;
 - recepción;
 - display;
 - checkout;
 - jornada;
 - beneficio;
 - coste;
 - slot;
 - backup.
-

24. Tutorial y ayuda

24.1. Principios

- enseñar una capacidad cuando se necesita;
- no bloquear exploración más de lo necesario;
- permitir repetir ayuda;
- no depender de una única ventana larga;
- registrar progreso;
- respetar cambios de bindings;
- no enseñar sistemas diferidos.

24.2. Tutorial inicial

Módulos:

1. movimiento;
2. cámara;
3. HUD;
4. Operations;
5. pedido;
6. recepción;
7. inventario;
8. display;
9. reposición;
10. apertura;
11. cliente;
12. checkout;
13. cierre;
14. resultados;
15. guardado.

24.3. Ayuda contextual

Ejemplos:

- primer placement inválido;
- primera falta de dinero;
- primera entrega;
- primer display vacío;
- primera cola;
- primer abandono;
- primer error de guardado.

24.4. Ayuda persistente

Operations debe incluir una sección consultable con:

- controles;
- flujos;
- conceptos;
- estados;
- preguntas frecuentes;
- accesibilidad.

25. Mensajes, feedback y diálogos

25.1. Niveles

Nivel	Uso
información	resultado ordinario
éxito	acción completada
advertencia	riesgo o estado no bloqueante

Nivel	Uso
error recuperable	acción rechazada o fallo corregible
error bloqueante	impide continuar
confirmación	decisión explícita
progreso	carga, guardado o proceso

25.2. Estructura de error

Un error útil contiene:

1. qué ocurrió;
2. por qué;
3. qué estado se conserva;
4. qué puede hacer la persona;
5. identificador técnico solo cuando sea apropiado.

25.3. Acciones destructivas

Requieren confirmación:

- borrar slot;
- reemplazar slot;
- salir sin guardar;
- retirar un mueble con contenido si produce consecuencias;
- cancelar un pedido cuando exista penalización;
- restablecer controles;
- restaurar valores de opciones.

25.4. Toasts

Adecuados para:

- guardado completado;
- pedido confirmado;

- transferencia completada;
- precio actualizado;
- venta;
- ayuda breve.

No adecuados para:

- corrupción de guardado;
- fallo de transición;
- pérdida de progreso;
- error que requiere una decisión.

25.5. Estados de carga

Un panel que carga datos muestra:

- estructura estable;
- indicador;
- bloqueo de acciones incompatibles;
- opción de reintentar si falla;
- estado vacío solo después de confirmar que no hay datos.

26. Fallos y recuperación

26.1. Guardado inválido

- no sobrescribir;
- intentar backup;
- informar;
- registrar;
- permitir volver.

26.2. Escena no cargable

- detener transición;
- mostrar error;
- volver a MainMenu o reintentar;
- evitar duplicar ApplicationRoot;
- conservar slot.

26.3. Estado incompatible

- detectar schema;
- migrar si existe ruta;
- rechazar de forma segura si no;
- explicar compatibilidad;
- no fingir carga exitosa.

26.4. Operación rechazada

Las operaciones de inventario, checkout, placement o economía:

- fallan sin mutación parcial;
- conservan selección cuando sea útil;
- explican causa;
- permiten corregir.

26.5. Cliente bloqueado

- timeout controlado;
- liberar reservas;
- registrar causa;
- retirar agente de forma segura;
- no bloquear cierre.

26.6. Cierre incompleto

Si existen procesos pendientes:

- mantener `Closing`;
- mostrar estado;
- aplicar fallback aprobado;
- no avanzar a resultados hasta estabilizar.

27. Flujos diferidos cercanos

27.1. Empleados

Flujo futuro:

```
Operations
→ Employees
→ revisar candidatos
→ comparar atributos y salario
→ contratar
→ asignar rol y horario
→ observar tarea en mundo
→ revisar rendimiento y cansancio
```

Reglas UX:

- no automatizar antes de comprender la tarea manual;
- mostrar prioridad;
- explicar inactividad;
- diferenciar falta de habilidad, descanso y bloqueo;
- confirmar despido;
- mostrar coste recurrente.

27.2. Investigación

```
Operations
→ Research
→ seleccionar rama
→ revisar requisitos
→ invertir
→ progresar
→ desbloquear
→ comunicar efecto
```

El desbloqueo debe conectar con una capacidad real y no ser una lista abstracta de porcentajes.

27.3. Puestos informáticos

```
BuildMode
→ colocar setup
→ instalar componentes
→ fijar tarifa
→ abrir servicio
→ cliente reserva
→ usa
→ paga
→ liberar
```

Debe reutilizar clientes, navegación, reservas, economía y cierre.

27.4. Reservas comerciales

Deben diferenciarse de la reserva lógica temporal de inventario.

Flujo posible:

```
cliente solicita reserva
→ comprobar política
→ apartar unidad
→ definir plazo
→ cobrar señal si aplica
→ recoger o caducar
```

28. Flujos empresariales futuros

28.1. Comercio online

```
activar canal  
→ publicar catálogo  
→ recibir pedido  
→ reservar stock  
→ picking  
→ packing  
→ seleccionar transporte  
→ enviar  
→ resolver incidencia
```

UX:

- inventario compartido visible;
- estados de pedido comprensibles;
- capacidad logística;
- alertas agregadas;
- no duplicar catálogo físico y online.

28.2. Publishing

```
recibir propuesta  
→ inspeccionar información e incertidumbre  
→ due diligence  
→ negociar  
→ firmar  
→ financiar hitos  
→ intervenir  
→ lanzar  
→ liquidar
```

Debe comunicar riesgo, capacidad editorial y retorno sin convertir todo en tablas opacas.

28.3. Desarrollo interno

```
detectar oportunidad
→ crear brief
→ prototipo
→ playtest
→ greenlight
→ producción por hitos
→ build
→ lanzamiento
```

Debe conservar vínculo con conocimiento del mercado y no sustituir el núcleo comercial.

28.4. Plataforma digital

```
planificar
→ asegurar catálogo
→ lanzar
→ adquirir usuarios
→ operar
→ moderar
→ medir reputación y rentabilidad
```

28.5. Infraestructura

Debe usar:

- pools de capacidad;
- módulos visibles;
- prioridades;
- degradación controlada;
- incidencias;
- mantenimiento.

28.6. Mercado y competidores

La UX debe mostrar:

- información parcial;
- tendencias;
- noticias;
- competidores visibles;
- reacciones;
- oportunidades;
- riesgos.

No debe revelar IA omnisciente ni producir presión incomprensible.

29. Arquitectura de pantallas

Pantalla o panel	Tipo	Estado
Bootstrap / transición	sistema	IMPLEMENTADO / revisar presentación
MainMenu	pantalla completa	IMPLEMENTADO
Slot Flow	pantalla o panel	IMPLEMENTADO
Options	pantalla o modal	IMPLEMENTADO parcial
Credits	pantalla	requerido
Loading	overlay o pantalla	requerido
HUD StoreInitial	overlay	IMPLEMENTADO / Sprint 16
Operations	panel principal	IMPLEMENTADO
Suppliers	panel	IMPLEMENTADO
Orders	panel	IMPLEMENTADO
Inventory	panel	IMPLEMENTADO
Displays	panel	IMPLEMENTADO
Customers	panel	IMPLEMENTADO
Checkout	panel/contextual	IMPLEMENTADO
Day	panel	IMPLEMENTADO

Pantalla o panel	Tipo	Estado
Economy	panel	IMPLEMENTADO
Help	panel	IMPLEMENTADO o pendiente de presentación
Accessibility	panel	IMPLEMENTADO base
BuildMode	panel + world overlay	IMPLEMENTADO
Pause	modal	IMPLEMENTADO
Daily Results	pantalla modal	IMPLEMENTADO
Error Recovery	modal/pantalla	requerido
Employees	panel	DIFERIDO
Research	panel	DIFERIDO
Computer Service	panel/contextual	DIFERIDO
Online	dashboard	VISIÓN FUTURA
Publishing	dashboard	VISIÓN FUTURA
Development	dashboard	VISIÓN FUTURA
Platform	dashboard	VISIÓN FUTURA
Infrastructure	dashboard	VISIÓN FUTURA
Market	dashboard	VISIÓN FUTURA

30. Criterios de aceptación UX

30.1. Arranque y escenas

ID	Criterio
UX-BOOT-001	La aplicación entra por Bootstrap.
UX-BOOT-002	MainMenu se carga mediante transición controlada.
UX-BOOT-003	No existen solicitudes concurrentes de escena.

ID	Criterio
UX-BOOT-004	TestLab no aparece en producción.
UX-BOOT-005	Un fallo de carga ofrece recuperación segura.
UX-BOOT-006	El clic de cierre de loading no mueve al jugador.

30.2. Slots

ID	Criterio
UX-SLOT-001	Un slot vacío ofrece nueva partida.
UX-SLOT-002	Un slot válido muestra información suficiente.
UX-SLOT-003	Reemplazar exige confirmación inequívoca.
UX-SLOT-004	Borrar exige confirmación.
UX-SLOT-005	El backup se ofrece cuando el primario falla.
UX-SLOT-006	Un error no sobrescribe el guardado existente.
UX-SLOT-007	La carga no finaliza antes de restaurar escena y estado.

30.3. Input

ID	Criterio
UX-INP-001	UI y mundo no procesan el mismo clic.
UX-INP-002	Solo un contexto captura input principal.
UX-INP-003	Escape resuelve primero el contexto más profundo.
UX-INP-004	Pausa impide mutaciones de gameplay.
UX-INP-005	Las instrucciones muestran bindings vigentes.
UX-INP-006	La cámara no activa interacciones.

30.4. Construcción

ID	Criterio
UX-BLD-001	Entrar en BuildMode suprime movimiento.
UX-BLD-002	Preview coincide con grid y huella.
UX-BLD-003	Q/E rota y revalida.
UX-BLD-004	Un placement inválido explica causa.
UX-BLD-005	Un placement inválido no cobra ni ocupa.
UX-BLD-006	Cancelar movimiento restaura estado anterior.
UX-BLD-007	Retirar no destruye stock silenciosamente.
UX-BLD-008	El feedback no depende solo del color.
UX-BLD-009	Cerrar BuildMode no produce clic de mundo.

30.5. Pedidos e inventario

ID	Criterio
UX-ORD-001	El pedido muestra total antes de confirmar.
UX-ORD-002	Saldo insuficiente se explica.
UX-ORD-003	La recepción no puede repetirse.
UX-INV-001	La UI distingue ubicaciones de stock.
UX-INV-002	Transferir valida origen, destino y cantidad.
UX-INV-003	Una transferencia falla sin estado parcial.
UX-INV-004	Los estados vacíos sugieren siguiente acción.
UX-DSP-001	Asignar producto no crea stock.
UX-DSP-002	Reasignar preserva unidades.
UX-DSP-003	Reponer actualiza lógica y visual.

30.6. Jornada y clientes

ID	Criterio
UX-DAY-001	El estado de jornada siempre es visible.
UX-DAY-002	Abrir valida bloqueos críticos.
UX-DAY-003	Closing bloquea nuevos clientes.
UX-DAY-004	Results espera resolución segura.
UX-CUS-001	El cliente comunica intención o problema.
UX-CUS-002	La paciencia se anticipa cuando es accionable.
UX-CUS-003	Abandono registra causa.
UX-CUS-004	Cierre libera reservas.
UX-CUS-005	Un cliente bloqueado no impide terminar jornada.

30.7. Checkout y economía

ID	Criterio
UX-CHK-001	Checkout muestra carrito y total.
UX-CHK-002	Preflight ocurre antes de mutar.
UX-CHK-003	Una venta se confirma una sola vez.
UX-CHK-004	Un fallo no mueve dinero ni stock.
UX-CHK-005	La cola avanza después de la venta.
UX-ECO-001	El saldo se actualiza de forma comprensible.
UX-ECO-002	Cada movimiento tiene causa consultable.
UX-ECO-003	Results separa ingresos, costes y beneficio.

30.8. Guardado y accesibilidad

ID	Criterio
UX-SAV-001	Guardar muestra progreso y resultado real.

ID	Criterio
UX-SAV-002	Un error de guardado permanece visible.
UX-SAV-003	Cargar conserva estado relevante.
UX-SAV-004	Salir con cambios ofrece opciones claras.
UX-ACC-001	Los estados críticos no dependen solo del color.
UX-ACC-002	La escala de UI es configurable.
UX-ACC-003	La velocidad de cámara es configurable.
UX-ACC-004	Los avisos importantes tienen alternativa visual.
UX-ACC-005	La lectura puede realizarse con simulación pausada.
UX-LOC-001	ES y EN no presentan truncamientos críticos.

30.9. Golden Path y build

ID	Criterio
UX-GP-001	El Golden Path se completa sin herramientas de Editor.
UX-GP-002	El recorrido se completa en StoreInitial.
UX-GP-003	Guardar y cargar conserva equivalencia observable.
UX-GP-004	La build Windows x64 reproduce el recorrido.
UX-GP-005	Player.log no contiene errores UX bloqueantes.
UX-GP-006	No existen defectos S0 o S1 abiertos.

31. Telemetría y playtest

31.1. Métricas recomendadas

- tiempo hasta MainMenu;
- tiempo hasta crear partida;

- tiempo hasta primer movimiento;
- tiempo hasta primer pedido;
- tiempo hasta primera recepción;
- tiempo hasta primera reposición;
- tiempo hasta apertura;
- tiempo hasta primera venta;
- porcentaje que completa primer día;
- porcentaje que completa Golden Path;
- aperturas de ayuda;
- errores de placement;
- clics de UI que generaron órdenes de mundo;
- abandonos por causa;
- errores de guardado;
- recuperación desde backup;
- tiempo en cada panel;
- acciones canceladas;
- pantallas con truncamiento o confusión observada.

31.2. Preguntas de playtest

- ¿La persona comprende qué hacer sin explicación externa?
- ¿Distingue almacén y exposición?
- ¿Comprende que asignar no repone?
- ¿Comprende por qué un cliente abandona?
- ¿Encuentra el cierre?
- ¿Interpreta correctamente Results?
- ¿Confía en el guardado?
- ¿Puede recuperarse de un error?
- ¿La tienda es legible desde cámara?
- ¿El panel Operations resulta centralizador o abrumador?

31.3. Evidencia

Cada playtest relevante debe registrar:

- build;
- configuración;
- participante;
- recorrido;
- duración;
- incidencias;
- citas o comentarios resumidos;
- métricas;
- severidad;
- decisión resultante.

32. Severidades UX

Severidad	Definición	Ejemplos
S0	pérdida grave, corrupción o bloqueo total	guardar destruye slot; no arranca
S1	Golden Path imposible	no puede abrir, vender o cerrar
S2	función importante degradada	panel inutilizable; error sin recuperación
S3	fricción significativa	feedback confuso; navegación inconsistente
S4	defecto menor	alineación, texto o pulido sin bloqueo

Gate:

- S0 abiertos: 0;
- S1 abiertos: 0;
- S2: resueltos o aceptados formalmente;
- S3/S4: clasificados y planificados.

33. Definition of Done de un flujo UX

Un flujo se considera cerrado cuando:

1. tiene entrada y salida definidas;
2. especifica propietario del input;
3. cubre éxito, cancelación, vacío, carga y error;
4. no produce mutación parcial;
5. presenta feedback comprensible;
6. no depende solo del color;
7. conserva foco y contexto;
8. puede operarse en la build objetivo;
9. dispone de criterios de aceptación;
10. tiene pruebas adecuadas;
11. está localizado;
12. no presenta truncamientos críticos;
13. se ha validado con el estado persistido;
14. no rompe otros recorridos;
15. actualiza documentación y trazabilidad.

34. Gates de Sprint 16 y Sprint 17

34.1. Sprint 16 — presentación representativa

Debe cerrar:

- StoreInitial autorada;
- lectura de entrada, mostrador, exposición y almacén;
- HUD coherente;

- feedback de construcción representativo;
- eliminación o aislamiento de marcadores técnicos;
- mobiliario y productos representativos;
- clientes y empleados con diferenciación visual cuando proceda;
- input UI/mundo corregido;
- coherencia entre visual y lógica.

34.2. Sprint 17 — estabilización

Debe cerrar:

- Golden Path;
- rendimiento;
- accesibilidad base;
- localización ES/EN;
- guardado y recuperación;
- QA completa;
- severidades;
- build Windows x64;
- validación de `Player.log`;
- evidencia;
- cierre documental.

34.3. Condición para iniciar sistemas posteriores

No iniciar empleados, investigación o puestos informáticos hasta que:

- el Golden Path sea reproducible;
- no existan S0/S1;
- StoreInitial esté en el flujo de build aprobado;
- los clics UI/mundo estén aislados;
- guardado/carga sea fiable;
- Results sea comprensible;
- exista una decisión formal de siguiente alcance.

Anexo A. Decisiones sustituidas

Materia	Decisión anterior	Regla vigente
Entrada	MainMenu podía considerarse punto directo	Bootstrap es única entrada
Flujo	Bootstrap → MainMenu → Store	Bootstrap → MainMenu → Slot Flow → StoreInitial → Day Flow
Tienda	5 × 5 m conceptual	aproximadamente 10 × 15 m, autorada
Estado	preproducción	vertical slice funcional integrado
Input	regla genérica	separación explícita UI/gameplay y puntero sobre UI
Guardado	Save/Load genérico	tres slots, backup y recuperación
Construcción	marcadores técnicos aceptables	feedback representativo requerido
Sistemas futuros	podían aparecer en mapa principal	ocultos hasta desbloqueo y gate

Anexo B. Trazabilidad de fuentes

Fuente	Líneas	Palabras aprox.	SHA-256	Uso
UX Flow v0.5	99	318	5c755541bbc69510af12839f8435e090972edc7b7bb484139683614f69fcb85c	autoridad actual de flujo, slots, Store, día e input
UX Flow v0.4	122	377	43143ab9e7504624ffdb4a2e9612332ee795f91ae05ea5d6448e5ce2f018bba8	flujo implementado previo y construcción
UX Flow v0.3 baseline v0.4	3202	12868	2f7e2355cb0e994394e536f8de514ba499aca16a2a81b8e6be1867a025842a9d	detalle extenso de recorridos y accesibilidad
UX Flow v0.3 baseline v0.3	3202	12853	abb29bbf9b9e9dfbef5161feb03ec07c7bc405cfe4b217e8843534279eb3b50c	trazabilidad histórica
ADR-0009	26	203	15408cc4633c35efc6f8b622230184e1636f9140898e48811a2f3498a0e0e2b5	Bootstrap, ApplicationRoot y transición de escenas
00_Enfoque_y_Alcance.md	4495	18925	63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f	autoridad consolidada superior
01_Game_Design_Document.md	4490	19760	a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177	autoridad consolidada superior
02_Vertical_Slice_Specification.md	1505	9834	75893f130116e88a08b8da5590dd81876a234581081eafaa808374c1a60513f	autoridad consolidada superior
03_Technical_Design_Document.md	3305	13774	66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12	autoridad consolidada superior
04_Modelo_de_Datos.md	2691	12944	d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4	autoridad consolidada superior

B.1. Comparación de las dos copias v0.3

Las copias de baseline v0.3 y v0.4 tienen ambas **3.202 líneas** y conservan la misma estructura funcional. Se detectaron **8 líneas de diferencia en el diff textual**, concentradas en metadatos y actualización del estado técnico. La copia de baseline v0.4 se utiliza como fuente extensa.

Anexo C. Regla de mantenimiento

El archivo debe guardarse como:

```
Documentacion/  
└─ 05_UX_Flow.md
```

Cuando cambie un recorrido:

1. actualizar este documento;
2. revisar criterios de aceptación;
3. comprobar TDD y modelo de datos;
4. revisar localización y accesibilidad;
5. actualizar pruebas;
6. registrar la decisión;
7. validar el flujo en StoreInitial y build.

Estado del documento: fuente vigente de experiencia de usuario y flujos para la nueva carpeta [Documentacion/](#).

Fin del documento.

Fuente: 05_UX_Flow.md · SHA-256: `e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e`

Diseño PDF: paleta VRM Games definida en la documentación de Cartridge & Cloud.